cria_func — Resumo Conceitual

INF1018 · Geração de código de máquina em tempo de execução · ideias e motivações

Este documento resume **por que** a `cria_func` é escrita do jeito que é. O foco não é cada byte ou cada linha, mas as ideias que justificam as decisões de projeto.

1. O que a função se propõe a fazer

A `cria_func` é uma **fábrica de funções**. Em vez de calcular um resultado, ela *fabrica* uma nova função em tempo de execução, escrevendo os bytes de instruções de máquina (x86-64) dentro de um vetor. Essa nova função é um **"trampolim"**: quando chamada, ela repassa o controle para uma função já existente `f`, mas com alguns argumentos preparados de antemão.

A ideia central é **"amarrar" parâmetros** — o conceito de aplicação parcial. A partir de uma função genérica como `mult(x, y)` podemos gerar, em tempo de execução, uma função especializada "multiplicar por 10" que recebe um só argumento e internamente chama `mult(x, 10)`.

> **Por que isso é possível?** Em C, um vetor de bytes e uma função são, no fundo, apenas endereços de memória. Se colocamos bytes que *são* instruções válidas e apontamos um ponteiro de função para o início do vetor, a CPU executa esses bytes como se fossem qualquer outra função. A `cria_func` apenas escreve os bytes certos.

2. De onde vem o valor de cada parâmetro

Cada parâmetro da função-alvo `f` precisa receber um valor. A descrição em `params[]` diz a **origem** de cada um — e é isso que torna a `cria_func` genérica em vez de servir a um único caso:

Origem	Ideia	Quando o valor é definido
`PARAM`	Repassa um argumento recebido pela própria função gerada	A cada chamada (vem de quem chama)
`FIX`	Uma constante amarrada, embutida no código	Na geração (congelado para sempre)
`IND`	Lê o valor de um endereço de memória	A cada chamada (lê o valor atual)

A distinção entre `FIX` e `IND` é conceitualmente importante: o `FIX` congela o valor no momento em que

a função é criada; o `IND` guarda um *endereço* e vai buscar o valor toda vez que a função é chamada — então o resultado pode mudar entre chamadas mesmo sem regenerar o código.

3. Por que existe um prólogo e um epílogo

A função gerada chama outra função (`f`). Isso a obriga a se comportar como uma função "bem-educada" perante a convenção de chamada: ela precisa montar um **stack frame** na entrada (prólogo) e desmontá-lo na saída (epílogo).

- **Prólogo** — salva o ponto-base do chamador, estabelece um quadro de referência estável e reserva espaço de trabalho na pilha.
- **Epílogo** — desfaz exatamente o que o prólogo fez, devolvendo a pilha ao estado original antes de retornar.

> **Por que isso não é opcional:** se a pilha não for desmontada simetricamente, a instrução de retorno volta para um endereço errado e o programa quebra. Prólogo e epílogo precisam estar sempre balanceados.

4. O cuidado com o alinhamento da pilha

A convenção de chamada (System V) **exige** que a pilha esteja alinhada a 16 bytes no momento de uma chamada. Funções reais assumem esse alinhamento; violá-lo causa falhas difíceis de diagnosticar. Por isso o espaço reservado na pilha é sempre arredondado para um múltiplo de 16 — não por capricho, mas porque é uma regra do ambiente que precisamos respeitar.

5. Por que salvar os argumentos na pilha antes de tudo

Este é o ponto mais sutil do projeto. Os argumentos chegam em registradores numa ordem fixa. Mas a ordem em que `f` os espera pode ser diferente — por exemplo, uma constante `FIX` pode ocupar o primeiro registrador, empurrando um argumento recebido para o segundo.

> **O problema:** se começássemos a preencher os registradores na ordem de `f`, poderíamos *sobrescrever* um argumento recebido antes de usá-lo — perdendo-o para sempre.

> **A solução:** antes de mexer em qualquer registrador, a função gerada **copia todos os**

> **argumentos recebidos para a pilha**. Com os valores guardados num lugar seguro, podemos então reorganizar os registradores livremente e buscar cada valor de volta na hora certa. É essa decisão que faz a reordenação de parâmetros funcionar corretamente.

6. Por que usar um registrador "de passagem"

Montar o valor de cada argumento primeiro num registrador de trabalho (`%rax`) e só depois copiá-lo para o registrador final é uma escolha de **simplicidade e uniformidade**. Os três casos (`PARAM`, `FIX`, `IND`) podem ser tratados da mesma forma — "produza o valor aqui" — e a etapa de "entregar no lugar certo" fica isolada e repetível. Isso reduz a quantidade de variações de código que precisamos gerar.

7. Por que o endereço de f é embutido no código

O endereço de `f` só é conhecido em tempo de execução (é um argumento da `cria_func`). Não dá para escrevê-lo como constante no programa-fonte; ele precisa ser **gravado dentro do código gerado** na hora. Esse é o mesmo princípio que vale para as constantes `FIX` e os endereços `IND`: cada instrução tem uma **parte fixa** (a operação) e uma **parte variável** (o operando) que a `cria_func` preenche dinamicamente. É essa separação que permite gerar código diferente para cada chamada.

8. Decisões de escopo e de estilo

- **Máximo de 3 parâmetros.** Limitar a 3 mantém tudo no terreno simples dos primeiros três registradores de argumento, evitando casos especiais de codificação e o trânsito de argumentos pela pilha. Uma verificação inicial protege esse contrato.
- **Tabelas de bytes com o assembly ao lado.** As sequências de bytes que dependem do registrador de destino ficam em tabelas comentadas com a instrução equivalente. Isso documenta a "tradução" e evita repetir bytes mágicos espalhados.
- **Pequenas funções auxiliares de emissão.** Centralizar o ato de "escrever bytes e avançar a posição" deixa o corpo principal legível como uma sequência de blocos (prólogo, preparação, chamada, epílogo), em vez de um amontoado de índices.

9. Em uma frase

> A `cria_func` é uma fábrica que escreve, em tempo de execução, o código de uma função-

> trampolim: ela monta um quadro de pilha seguro, guarda os argumentos recebidos, coloca em cada registrador o valor certo (recebido, constante ou lido da memória) respeitando a convenção de chamada, invoca `f` e devolve o que `f` retornou — efetivamente criando uma versão especializada de `f`.